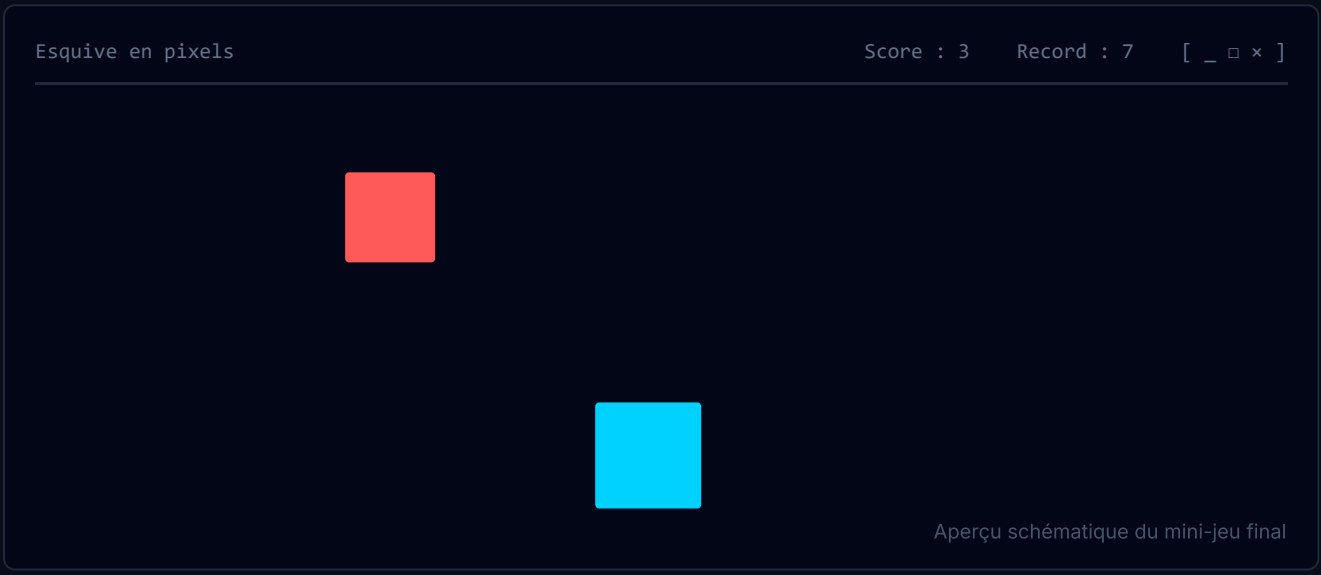
Aucune expérience en programmation nécessaire

ESQUIVE EN PIXELS

Crée ton premier mini-jeu vidéo 2D, une mission à la fois.



Aperçu schématique du mini-jeu final

Ton objectif

Déplace le vaisseau bleu, esquive l'astéroïde rouge, accumule des points et tente de battre le record de session.

Ton résultat

Un jeu fluide et réactif, un code source Python de référence propre et tous les outils pour dépanner en autonomie.

BIENVENUE À BORD

Ta feuille de route

Ce guide t'accompagne pas à pas. Chaque mission s'appuie sur la précédente sans jamais sauter d'étape.

03 Le projet et ses règles
Ce que tu construis et comment jouer

04 Installer Python
L'environnement officiel x86-64

05 Préparer VS Code
Le dossier et l'espace de code

06 Installer Pygame & Venv
Environnement isolé et interpréteur

07 Les repères essentiels
Variables, coordonnées et boucle

08–10 Fenêtre, Joueur, Obstacle
Missions 1 à 3 : premières briques

11–13 Score, Collision, Fin
Missions 4 et 5 : règles du jeu

14–16 Redémarrage & Vitesse
Missions 6 et 7 : boucle complète

17–18 Code complet de référence
Programme complet assemblé

19 Architecture & Fonctionnement
Comprendre la mécanique interne

20–21 Boîte à outils de dépannage
Résoudre erreurs et blocages

22–25 Mémo, Défis & Mentions
Aller plus loin et informations d'édition

La méthode la plus fiable

1. Lis la consigne jusqu'au bout.
2. Écris le code sans copier-coller à l'aveugle.
3. Sauvegarde avec **Ctrl + S**.
4. Teste l'exécution et observe le terminal.

Configuration système

Conçu pour le terminal **PowerShell** sous **Windows 11 ou Windows 10 (64 bits)**. Garde cette fenêtre de guide à côté de ton écran d'éditeur.

AVANT DE COMMENCER

Une mission à ta portée

Le jeu est un défi d'arcade en 2D en temps réel. Tu diriges un carré bleu clair au bas de l'écran pour éviter un carré rouge qui chute à vitesse croissante.

Les règles du jeu

- L'obstacle tombe verticalement depuis le haut.
- Sortir par le bas sans collision = +1 point.
- Si l'obstacle chevauche le joueur = Game Over.
- La touche **R** relance instantanément la manche.

Score & Record

- Le score revient à 0 après chaque défaite.
- Le record garde le score le plus élevé de la session.
- Le record persiste après redémarrage avec **R**.
- Fermer la fenêtre remet le record à zéro.

Les commandes clavier

Touche	Action
← Flèche Gauche	Déplacer le joueur vers la gauche
→ Flèche Droite	Déplacer le joueur vers la droite
R	Relancer une manche après un « GAME OVER »
× Croix de fenêtre	Quitter proprement le programme

Matériel & Environnement recommandé

Un ordinateur sous **Windows 11 ou Windows 10 (64-bit architecture x86-64)**. *Note système :* le support standard de Windows 10 ayant pris fin le 14 octobre 2025, nous conseillons un système régulièrement mis à jour pour garantir la sécurité globale de ta machine.

ÉTAPE 1

Installer Python

Python est le moteur qui exécutera ton jeu. Suis précisément ces instructions pour installer la version officielle recommandée.

1 Télécharger l'installateur officiel Windows

Rends-toi sur la page officielle des versions Windows :
https://www.python.org/downloads/windows/

> **Vérification capitale du fichier :** Choisis impérativement le lien **« Windows installer (64-bit) »** (architecture x86-64) pour Python 3.13.x. N'installe *pas* la version ARM64, ni les paquets "embeddable zip", ni les versions expérimentales dites "free-threaded".

2 Lancer et paramétrer l'installation

Ouvre le fichier téléchargé. **Coche impérativement la case « Add python.exe to PATH »** tout en bas de la fenêtre d'accueil avant de cliquer sur **« Install Now »**. Patiente jusqu'au message de confirmation.

3 Vérifier l'installation dans l'Invite de commandes

Ouvre le menu Démarrer de Windows, tape `cmd` et appuie sur Entrée. Saisis :

```
py -3.13 --version
```

Tu dois obtenir une réponse confirmant `Python 3.13.x`. Si ce message s'affiche, Python est prêt !

ÉTAPE 2

Préparer VS Code

Visual Studio Code est l'éditeur de texte professionnel dans lequel tu vas structurer et coder ton jeu.

1 Installer l'éditeur

Télécharge VS Code depuis code.visualstudio.com, lance l'installateur avec les options par défaut, puis démarre l'application.

2 Créer le dossier de travail

1. Sur ton Bureau Windows, crée un nouveau dossier nommé `MissionPython`.
2. Dans VS Code, clique sur **Fichier** → **Ouvrir le dossier** (ou *File* → *Open Folder*).
3. Sélectionne le dossier `MissionPython`. Si VS Code te demande si tu fais confiance aux auteurs, confirme en cliquant sur *« Yes, I trust the authors »*.

3 Créer le fichier principal

Dans l'explorateur à gauche, clique sur l'icône de nouveau fichier et saisis exactement :

```
jeu.py
```

Attention : assure-toi que l'extension est bien `.py` et non `jeu.py.txt`.

Extension recommandée

Ouvre l'onglet Extensions (raccourci **Ctrl + Maj + X**), cherche **Python** publié par *Microsoft* et installe-la. Elle t'assure la coloration syntaxique et le repérage visuel des erreurs.

ÉTAPE 3

Installer Pygame dans un environnement isolé

Pygame gère l'affichage, les formes et la lecture du clavier. Nous allons l'installer dans un environnement virtuel propre au projet (`.venv`).

1 Ouvrir le terminal intégré

Dans VS Code, rends-toi dans le menu **Terminal** → **Nouveau terminal**. Vérifie que le prompt indique bien ton dossier `MissionPython`.

2 Créer l'environnement virtuel

```
py -3.13 -m venv .venv
```

Action essentielle :
Dans VS Code, ouvre la palette de commandes avec **Ctrl + Maj + P**, tape `Python: Select Interpreter`, valide, puis clique sur l'interpréteur situé dans ton dossier local : `.\.venv\Scripts\python.exe`. Ton éditeur est désormais parfaitement synchronisé avec ton projet.

3 Installer Pygame 2.6.1 de manière déterministe

```
.\.venv\Scripts\python.exe -m pip install pygame==2.6.1
.\.venv\Scripts\python.exe -c "import pygame; print(pygame.version.ver)"
```

La commande doit répondre **2.6.1**. Ton atelier de développement est fin prêt !

COMPRENDRE AVANT DE CODER

Les 4 piliers de notre programme

1. Variables & Affectation

Associe une valeur modifiable à une étiquette :

```
score = 0
vitesse = 5
```

2. Conditions de contrôle

Déclenche une action si le test est validé :

```
if score == 10:
    print("Bravo !")
```

3. Le piège = contre ==

- = **attribue** une valeur dans une variable.
- == **compare** deux éléments (vrai ou faux).

4. Indentation stricte

Python délimite ses blocs par **4 espaces**. N'utilise jamais de tabulations mélangées.

Le système de coordonnées 2D

L'origine `(0, 0)` se situe dans l'angle **supérieur gauche** de l'écran. Augmenter x fait glisser l'objet vers la droite ; augmenter y fait descendre l'objet vers le bas. Les rectangles sont gérés via `pygame.Rect(x, y, largeur, hauteur)`.

MISSION 1

Ouvrir une fenêtre de jeu

Ouvre `jeu.py` dans VS Code et écris l'ossature fondamentale du moteur :

```
import pygame
import random

pygame.init()
LARGEUR = 800
HAUTEUR = 600
fenetre = pygame.display.set_mode((LARGEUR, HAUTEUR))
pygame.display.set_caption("Esquive en pixels")
horloge = pygame.time.Clock()

# VARIABLES DU JEU

while True:
    # ÉVÉNEMENTS
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            raise SystemExit

    # DÉPLACEMENTS ET COLLISIONS

    # AFFICHAGE
    fenetre.fill((20, 30, 60))
    pygame.display.flip()
    horloge.tick(60)
```

Exécution : Enregistre avec `Ctrl + S` puis lance dans le terminal :

```
.\.venv\Scripts\python.exe jeu.py
```

Une fenêtre bleu nuit apparaît et tourne à 60 images par seconde. Ferme-la avec la croix.

MISSION 2

Créer et déplacer le joueur

Nous ajoutons un rectangle bleu ciel contrôlable au clavier, confiné dans l'écran.

1. **Créer le rectangle du joueur** (avant `while True:`, sans indentation) :

```
joueur = pygame.Rect(375, 520, 50, 50)
vitesse_joueur = 5
```

2. **Écouter les flèches du clavier** (sous `# DÉPLACEMENTS ET COLLISIONS` avec 4 espaces) :

```
    touches = pygame.key.get_pressed()
    if touches[pygame.K_LEFT]:
        joueur.x -= vitesse_joueur
    if touches[pygame.K_RIGHT]:
        joueur.x += vitesse_joueur
    if joueur.left < 0:
        joueur.left = 0
    if joueur.right > LARGEUR:
        joueur.right = LARGEUR
```

3. **Dessiner le joueur** (dans `# AFFICHAGE`, après `fenetre.fill`) :

```
    pygame.draw.rect(fenetre, (0, 220, 255), joueur)
```

Test interactif : Relance le jeu. Clique sur la fenêtre pour lui donner le focus : les touches fléchées gauche et droite déplacent le carré bleu sans qu'il ne sorte jamais de l'écran.

MISSION 3

Faire chuter l'obstacle

Nous créons maintenant un carré rouge qui descend de façon continue et réapparaît aléatoirement en haut.

1. **Déclarer l'obstacle avec la constante de vitesse** (avant `while True:`) :

```
obstacle = pygame.Rect(200, 0, 40, 40)
VITESSE_OBSTACLE_INITIALE = 3
vitesse_obstacle = VITESSE_OBSTACLE_INITIALE
```

2. **Programmer la chute et le recyclage** (dans les déplacements) :

```
    obstacle.y += vitesse_obstacle
    if obstacle.top >= HAUTEUR:
        obstacle.y = 0
        obstacle.x = random.randint(0, LARGEUR - obstacle.width)
```

3. **Dessiner l'obstacle** (dans la section affichage) :

```
    pygame.draw.rect(fenetre, (255, 90, 90), obstacle)
```

> **Note pédagogique :** Pour l'instant, l'obstacle traverse le joueur sans effet. La détection physique du contact sera implémentée lors de la mission 5.

MISSION 4

Afficher le score et le record

Chaque obstacle qui traverse l'écran sans collision rapporte un point.

1 **Initialiser le compteur et la typographie** (avant `while True:`) :

```
score = 0
record = 0
police_score = pygame.font.Font(None, 36)
```

2 **Incrémenter le score au bon moment** :

```
if obstacle.top >= HAUTEUR:
    obstacle.y = 0
    obstacle.x = random.randint(0, LARGEUR - obstacle.width)
    score += 1
    if score > record:
        record = score
```

3 **Dessiner le texte à l'écran** (avant `pygame.display.flip()`) :

```
texte_score = police_score.render(
    f"Score : {score}  Record : {record}", True, (255, 255, 255)
)
fenetre.blit(texte_score, (20, 20))
```

MISSION 5 · PARTIE A

Détecter le chevauchement (Collision)

Quand l'obstacle percute le vaisseau, le jeu doit s'arrêter net.

1. **Variable d'état de partie** (avant `while True:`) :

```
jeu_termine = False
```

2. **Conditionner les déplacements** :

> **Précision importante :** Le bloc ci-dessous remplace tout le contenu de `# DÉPLACEMENTS ET COLLISIONS` et s'arrête strictement *juste avant* le commentaire `# AFFICHAGE`. Conserve bien la section d'affichage en dehors de ce bloc.

```
# DÉPLACEMENTS ET COLLISIONS
if not jeu_termine:
    touches = pygame.key.get_pressed()
    if touches[pygame.K_LEFT]:
        joueur.x -= vitesse_joueur
    if touches[pygame.K_RIGHT]:
        joueur.x += vitesse_joueur
    if joueur.left < 0:
        joueur.left = 0
    if joueur.right > LARGEUR:
        joueur.right = LARGEUR

    obstacle.y += vitesse_obstacle
    if joueur.colliderect(obstacle):
        jeu_termine = True
    elif obstacle.top >= HAUTEUR:
        obstacle.y = 0
        obstacle.x = random.randint(0, LARGEUR - obstacle.width)
        score += 1
        if score > record:
            record = score
```

MISSION 5 · PARTIE B

Afficher l'écran de fin de partie

Nous prévenons le joueur avec un texte centré et visible dès que la défaite survient.

1. **Créer la police grand format** (avant `while True:`) :

```
police_fin = pygame.font.Font(None, 60)
```

2. **Affichage conditionnel centré** (dans `# AFFICHAGE`) :

```
if jeu_termine:
    texte_fin = police_fin.render("GAME OVER", True, (255, 255, 255))
    rectangle_fin = texte_fin.get_rect(
        center=(LARGEUR // 2, HAUTEUR // 2 - 20)
    )
    fenetre.blit(texte_fin, rectangle_fin)
```

Comprendre le centrage : `LARGEUR // 2` effectue une division entière exacte pour calculer l'axe médian de la fenêtre sans générer de nombre flottant.

MISSION 6

Recommencer instantanément avec la touche R

Permettons de relancer une manche sans quitter la fenêtre, tout en garantissant la cohérence de la vitesse.

1. **Préparer la police d'indication** (avant `while True:`) :

```
police_aide = pygame.font.Font(None, 28)
```

2. **Intercepter la touche R dans les événements** :

```
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_r and jeu_termine:
            joueur.topleft = (375, 520)
            obstacle.x = random.randint(0, LARGEUR - obstacle.width)
            obstacle.y = 0
            score = 0
            vitesse_obstacle = VITESSE_OBSTACLE_INITIALE  # Réinitialisation de la vitesse
            jeu_termine = False
```

3. **Afficher la consigne textuelle** (dans `if jeu_termine:`) :

```
    texte_aide = police_aide.render(
        "Appuie sur R pour recommencer", True, (255, 255, 255)
    )
    rectangle_aide = texte_aide.get_rect(
        center=(LARGEUR // 2, HAUTEUR // 2 + 35)
    )
    fenetre.blit(texte_aide, rectangle_aide)
```

MISSION 7

Accélérer l'action (Difficulté croissante)

Tous les 5 points marqués, la chute s'accélère pour intensifier la partie, avec un plafond maximal fixé à 12.

Dans la condition `elif obstacle.top ⩾ HAUTEUR:`, ajoute la règle de palier :

```
    elif obstacle.top >= HAUTEUR:
        obstacle.y = 0
        obstacle.x = random.randint(0, LARGEUR - obstacle.width)
        score += 1
        if score > record:
            record = score
        if score % 5 == 0 and vitesse_obstacle < 12:
            vitesse_obstacle += 1
```

Comment fonctionne le modulo % ?

L'opérateur `score % 5` calcule le reste de la division par 5. Dès que le score atteint 5, 10, 15… le reste est strictement 0, ce qui déclenche l'accélération. Le plafond de 12 est atteint à 45 points.

PERSONNALISATION & VÉRIFICATIONS

Rends ton jeu unique

Règle d'indentation capitale :
Les lignes ci-dessous remplacent des instructions existantes. Conserve impérativement leur indentation d'origine : **4 espaces** pour le fond et les rectangles dans la boucle de jeu ; **0 espace** pour les variables définies avant la boucle.

Fond d'écran violet spatial

```
    fenetre.fill((30, 15, 45))
```

Couleur vaisseau vert émeraude

```
    pygame.draw.rect(fenetre, (50, 255, 100), joueur)
```

Changer le titre de la fenêtre

```
pygame.display.set_caption("Défi Galactique")
```

Ajuster la vitesse de départ

```
VITESSE_OBSTACLE_INITIALE = 5
vitesse_obstacle = VITESSE_OBSTACLE_INITIALE
```

FICHIER COMPLET JEU.PY

Partie 1 : Initialisation & Événements

Ce fragment forme le début du fichier unique `jeu.py`. (Ne lance pas le jeu avant d'avoir collé la suite page 18).

```
import pygame
import random

pygame.init()
LARGEUR = 800
HAUTEUR = 600
fenetre = pygame.display.set_mode((LARGEUR, HAUTEUR))
pygame.display.set_caption("Esquive en pixels")
horloge = pygame.time.Clock()

# VARIABLES DU JEU
joueur = pygame.Rect(375, 520, 50, 50)
vitesse_joueur = 5

obstacle = pygame.Rect(200, 0, 40, 40)
VITESSE_OBSTACLE_INITIALE = 3
vitesse_obstacle = VITESSE_OBSTACLE_INITIALE

score = 0
record = 0
police_score = pygame.font.Font(None, 36)

jeu_termine = False
police_fin = pygame.font.Font(None, 60)
police_aide = pygame.font.Font(None, 28)

while True:
    # ÉVÉNEMENTS
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            raise SystemExit
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r and jeu_termine:
                joueur.topleft = (375, 520)
                obstacle.x = random.randint(0, LARGEUR - obstacle.width)
                obstacle.y = 0
                score = 0
                vitesse_obstacle = VITESSE_OBSTACLE_INITIALE
                jeu_termine = False
```

FICHIER COMPLET JEU.PY

Partie 2 : Déplacements & Rendu

À coller immédiatement à la suite de la page 17, dans le même fichier, aligné à 4 espaces.

```
    # DÉPLACEMENTS ET COLLISIONS
    if not jeu_termine:
        touches = pygame.key.get_pressed()
        if touches[pygame.K_LEFT]:
            joueur.x -= vitesse_joueur
        if touches[pygame.K_RIGHT]:
            joueur.x += vitesse_joueur
        if joueur.left < 0:
            joueur.left = 0
        if joueur.right > LARGEUR:
            joueur.right = LARGEUR

        obstacle.y += vitesse_obstacle
        if joueur.colliderect(obstacle):
            jeu_termine = True
        elif obstacle.top >= HAUTEUR:
            obstacle.y = 0
            obstacle.x = random.randint(0, LARGEUR - obstacle.width)
            score += 1
            if score > record:
                record = score
            if score % 5 == 0 and vitesse_obstacle < 12:
                vitesse_obstacle += 1

    # AFFICHAGE
    fenetre.fill((20, 30, 60))
    pygame.draw.rect(fenetre, (0, 220, 255), joueur)
    pygame.draw.rect(fenetre, (255, 90, 90), obstacle)

    texte_score = police_score.render(
        f"Score : {score}  Record : {record}", True, (255, 255, 255)
    )
    fenetre.blit(texte_score, (20, 20))

    if jeu_termine:
        texte_fin = police_fin.render("GAME OVER", True, (255, 255, 255))
        rectangle_fin = texte_fin.get_rect(center=(LARGEUR // 2, HAUTEUR // 2 - 20))
        fenetre.blit(texte_fin, rectangle_fin)

        texte_aide = police_aide.render(
            "Appuie sur R pour recommencer", True, (255, 255, 255)
        )
        rectangle_aide = texte_aide.get_rect(center=(LARGEUR // 2, HAUTEUR // 2 + 35))
        fenetre.blit(texte_aide, rectangle_aide)

    pygame.display.flip()
    horloge.tick(60)
```

ANALYSE TECHNIQUE

Comment le moteur tourne en mémoire

01 · Initialisation

Création des surfaces vidéo, paramétrage de l'horloge et allocation des objets géométriques et typographiques.

02 · Collecte d'événements

Vidage de la file d'attente système : surveillance de la croix de fermeture et de la touche R.

03 · Simulation physique

Si la manche est active : lecture continue des touches, mise à jour des positions, test d'intersection AABB (colliderect).

04 · Rendu graphique

Nettoyage de l'écran (fill), dessin des rectangles, projection des textes, et basculement du tampon mémoire (flip).

Tableau des variables d'état

Variable	Rôle en jeu	Effet lors de la touche R
`score`	Points de la manche en cours	Remis à 0
`record`	Meilleur score de la session	Conservé intact
`vitesse_obstacle`	Rapidité de la chute	Réinitialisée à la constante

DÉPANNAGE

Résoudre les problèmes d'environnement

1 · « py » n'est pas reconnu dans le terminal

Ferme le terminal, rouvre-le. Si l'erreur persiste, réinstalle Python en cochant impérativement **« Add python.exe to PATH »**.

2 · « Python 3.13 est introuvable »

Vérifie l'architecture Windows téléchargée. Tu dois impérativement utiliser **l'installateur Windows x86-64** et non la mouture ARM64.

3 · « No module named pygame »

Pygame n'a pas été installé dans l'interpréteur actif. Relance la commande en pointant directement le binaire du venv :

```
.\.venv\Scripts\python.exe -m pip install pygame==2.6.1
```

Attention aux noms interdits : Ne nomme jamais ton script `pygame.py` ou `random.py`, sous peine d'écraser les bibliothèques officielles de Python. Utilise toujours `jeu.py`.

DÉPANNAGE

Résoudre les anomalies de code

SyntaxError

Oubli de deux-points `:`, de parenthèse ou guillemet non refermé. Regarde la ligne indiquée et celle juste au-dessus.

IndentationError / TabError

Mélange d'espaces et de tabulations. Dans VS Code, active l'affichage des espaces pour vérifier l'alignement sur 4 espaces.

NameError

Une variable est mal orthographiée. Attention : Python est sensible à la casse (`Score` ≠ `score`).

La fenêtre s'ouvre et se ferme

Le script a rencontré une erreur dès le lancement. Lis attentivement les dernières lignes affichées en rouge dans le terminal.

La méthode d'investigation en 3 étapes

1. Repère le type d'erreur et le numéro de ligne.
2. Compare avec le code de référence des pages 17 et 18.
3. Corrige un seul élément à la fois, enregistre avec **Ctrl + S** et relance.

LEXIQUE TECHNIQUE

Ton mémo de développeur

Terme	Définition exacte	Exemple dans notre jeu
`Variable`	Emplacement mémoire nommé stockant une valeur	`score = 0`
`Boucle while`	Exécute des instructions en continu tant qu'une condition est vraie	`while True:`
`Rect`	Structure mathématique représentant coordonnées et dimensions	`pygame.Rect(x,y,w,h)`
`Collision`	Chevauchement géométrique de surfaces rectangulaires	`colliderect()`
`Tick (60)`	Limite le rafraîchissement à 60 boucles par seconde au maximum	`horloge.tick(60)`

Opérateurs Python indispensables

- `+= 1` : Incrémente la variable de 1.
- `-= 1` : Décrémente la variable de 1.
- `and` / `not` : Opérateurs logiques d'association et d'inversion.
- `%` (modulo) : Calcule le reste entier de la division.

PROGRESSION

Trois défis pour aller plus loin

1. Personnaliser le graphisme (Facile)

Modifie les dimensions des rectangles ou adopte une palette de couleurs contrastées (triplets RVB de 0 à 255).

2. Ajouter un second obstacle (Intermédiaire)

Crée `obstacle2`. Attention : n'oublie pas de gérer sa chute, sa collision, et sa réinitialisation complète lors de l'appui sur R !

3. Le défi expert : Déplacements en temps réel (Delta Time)

Encadrement strict : Pour rendre la vitesse indépendante des variations de FPS :
• Récupère les secondes écoulées avec `dt = horloge.tick(60) / 1000.0` (un seul appel par image en début de boucle !).
• Exprime la vitesse en pixels par seconde (ex: 300 px/s).
• Stocke la position dans des variables flottantes séparées (ex: `joueur_x += vitesse * dt`) avant de l'assigner au rectangle entier.

RECETTE DE VALIDATION

Checklist de fin de projet

- [ ] La fenêtre s'ouvre en 800×600 sans crash.
- [ ] Le vaisseau ne traverse pas les bords d'écran.
- [ ] L'obstacle réapparaît à une abscisse aléatoire.
- [ ] Les points augmentent lors d'une esquive réussie.
- [ ] Le contact déclenche le GAME OVER.
- [ ] La touche R remet le score à zéro et relance le jeu.
- [ ] Le record de session reste affiché après R.
- [ ] La vitesse accélère par palier de 5 points.

Sauvegarder son projet proprement

Ne copie **jamais** le dossier `.venv` sur tes clés USB ou ton cloud : il est volumineux et lié à ta machine. Sauvegarde uniquement tes sources : `jeu.py` et un fichier texte `requirements.txt` contenant la ligne `pygame==2.6.1`. L'environnement se recrée en 10 secondes avec la commande vue en page 6.

DOCUMENTATION & PROLONGEMENTS

Continuer à apprendre

Tutoriel Python Officiel (3.13)

docs.python.org/fr/3.13/tutorial/

Documentation Pygame

pygame.org/docs/

INFORMATIONS D'ÉDITION ET DROITS D'USAGE

Titre : Esquive en Pixels — Guide Pratique Débutant (Édition Windows)
Autoéditeur : Quanthevaris, France.
Diffusion : Publication numérique gratuite non périodique — Version 1.0 (Octobre 2026).
Contact : contact.quanthevaris@gmail.com
Politique de confidentialité : quanthevaris.github.io
ISBN : Non applicable (publication numérique gratuite hors commerce, dispensée de numéro obligatoire selon l'AFNIL et la BnF).

Propriété intellectuelle & Licences :
• *Code source Python :* Mis à disposition sous licence libre permissive **MIT** (texte officiel complet disponible sur opensource.org/licenses/MIT) : libre d'utilisation, copie, modification et réutilisation commerciale ou non commerciale pour tes propres jeux.
• *Guide pédagogique (texte, maquette et illustrations) :* © 2026 Tous droits réservés. Usage strictement personnel et éducatif. La redistribution ou la vente du guide sans accord préalable de l'auteur est interdite.
• *Indépendance des marques :* Python, Visual Studio Code, Pygame, Microsoft Corporation et Windows sont des marques déposées de leurs éditeurs respectifs. Ce guide est un projet pédagogique indépendant et n'a fait l'objet d'aucune affiliation ni approbation de la part de ces organisations.

Clause de non-responsabilité :
Ce guide pédagogique est fourni « en l'état », à titre purement informatif et éducatif, sans garantie d'aucune sorte, explicite ou implicite. L'auteur décline toute responsabilité quant aux éventuelles erreurs, omissions ou imprécisions contenues dans ce document, ainsi qu'aux conséquences directes ou indirectes résultant de l'installation des logiciels mentionnés ou de l'exécution du code source fourni, y compris toute perte de données, dysfonctionnement matériel ou logiciel survenant sur le système de l'utilisateur. L'utilisation des informations et du code de ce guide se fait sous l'entière responsabilité du lecteur.